



PROJECT REPORT

HOSTELHUB

HOSTEL MANAGEMENT SYSTEM

PROGRAMMING IN JAVA

Course Code: CSE2006

Project Category: VITYarthi "Build Your Own Project"

Academic Term: Fall Semester 2026

SUBMITTED BY

Name: Aryan Chirag

Registration Number: 25BAI11075

Program: B.Tech Computer Science and Engineering (AI&ML;)

School / Department: SCAI

Institution: VIT Bhopal University, Madhya Pradesh, India

EVALUATED BY

Faculty Mentor: Dr. Adarsh Patel

Course Code: CSE2006

Slot: C11+C12

TABLE OF CONTENTS

1.	Executive Summary	3
2.	Introduction	4
3.	Problem Statement	4
4.	Functional Requirements	5
5.	Non-functional Requirements	6
6.	System Architecture	7
7.	Design Diagrams	8
7.1	Use Case Diagram	8
7.2	Workflow Diagram	9
7.3	Sequence Diagram	10
7.4	Class Diagram	11
7.5	Storage / ER-style Diagram	12
8.	Design Decisions & Rationale	13
9.	Implementation Details	14
10.	Screenshots / Results	15
11.	Testing Approach	18
12.	Challenges Faced	19
13.	Learnings & Key Takeaways	19
14.	Future Enhancements	20
15.	References	20

1. Executive Summary

HostelHub is a console-based hostel management system developed in Java for managing common hostel operations in one application. It maintains student records, room allocation and occupancy, complaints, visitor entries and exits, leave requests, and basic hostel analytics.

The project applies Java concepts including classes and objects, encapsulation, constructors, conditional statements, loops, switch-based menu navigation, Java Collections using ArrayList, file input/output, and exception handling. Records are persisted locally using CSV files so saved information can be loaded again when the application starts.

The application separates interaction from core operations. HostelSystem handles menus, navigation and input, while HostelServices performs record operations, searches, analytics and CSV persistence. Five model classes represent the main records.

Project at a Glance

- **Application:** Menu-driven Java console application
- **Storage:** Local CSV files
- **Major modules:** Student, Room, Complaint, Visitor, Leave and Analytics
- **Testing:** HostelHubTest.java

2. Introduction

Hostel administration involves several related records. Student information must be associated with room assignments, rooms must reflect occupancy, complaints need status tracking, visitor entries require entry and exit information, and leave requests require status updates.

HostelHub provides a simple local solution for these operations. A user selects a module from the command-line menu, enters the required information, receives validation feedback, and returns to the main menu. CSV files provide local persistence without requiring an external database server.

3. Problem Statement

Maintaining hostel information manually or through separate records can make it difficult to retrieve and update student details, room allocation information, complaints, visitor records and leave requests in a consistent manner.

HostelHub addresses this by providing one application for these routine hostel operations. The system focuses on organized record management, basic validation, persistent local storage and summary analytics.

Objectives

- Maintain student records in an organized form.
- Manage room capacity and occupancy.
- Record and resolve hostel complaints.
- Maintain visitor entry and exit information.
- Submit and update student leave requests.
- Provide basic hostel statistics.
- Apply Java and object-oriented programming concepts.
- Use Java file handling for persistent local storage.

4. Functional Requirements

HostelHub contains six major functional modules, exceeding the minimum three-module requirement in the VITyarthi instructions.

4.1 Student Management

- Add, view, search and remove student records.
- Prevent duplicate student IDs.
- Check that the selected room exists and has capacity.
- Update room occupancy when a student is removed.

4.2 Room Management

- Add rooms and validate capacity.
- View rooms and available rooms.
- Track occupied and available beds.

4.3 Complaint Management

- Submit complaints for existing students.
- Store category, description, priority and status.
- View and resolve complaints.

4.4 Visitor Management

- Register visitors against existing students.
- Store visitor details and entry/exit information.
- View visitors and record exits.

4.5 Leave Management

- Submit requests for existing students.
- Store reason and dates.
- View requests and approve or reject them.

4.6 Analytics

- Display record totals and occupied/available beds.

5. Non-functional Requirements

5.1 Usability

The application uses numbered menus, clear prompts and direct feedback. Users can return to the main menu after completing an operation.

5.2 Performance

The application is designed for local hostel records. ArrayList provides simple in-memory collection handling and CSV files are accessed locally.

5.3 Reliability

Related operations check whether referenced records exist before proceeding. Room allocation checks capacity, while complaint, visitor and leave operations verify the associated student.

5.4 Data Persistence

Data-changing operations save the current collections to CSV files. At startup, stored records are loaded into the corresponding Java collections.

5.5 Maintainability

The application separates the entry point, user interaction, service operations and model classes, reducing the amount of logic concentrated in one class.

5.6 Error Handling

Input methods validate numeric choices and required text, while file operations handle loading and saving failures without exposing unnecessary implementation details.

6. System Architecture

HostelHub follows a simple layered structure suitable for a console-based Java application. `HostelSystem.java` manages presentation and interaction, `HostelServices.java` manages core operations and persistence, model classes represent application records, and CSV files provide local storage.

HostelHub Architecture

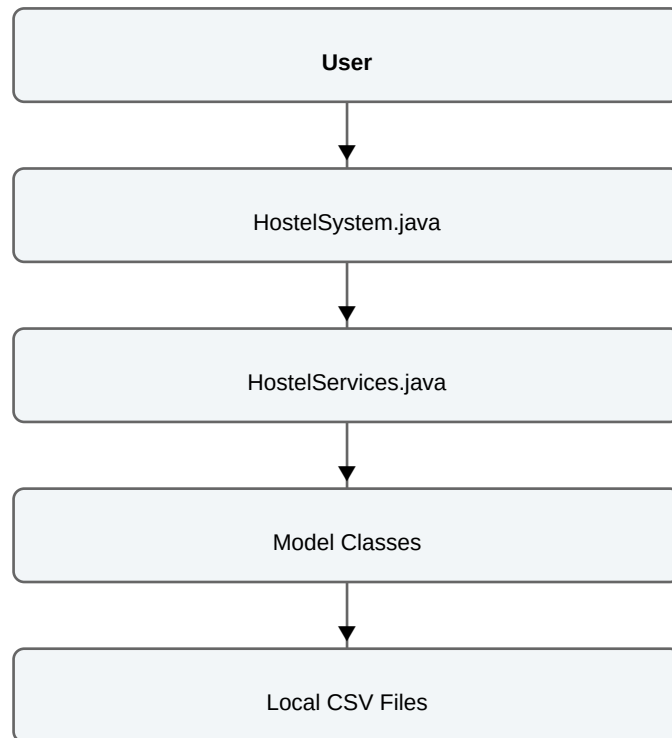


Figure 1. HostelHub system architecture and data flow.

Main Components

- **Main.java:** Starts the application.
- **HostelSystem.java:** Menu navigation, input and validation.
- **HostelServices.java:** Core operations, searching, analytics and CSV loading/saving.
- **Model classes:** Student, Room, Complaint, Visitor and LeaveRequest.
- **CSV storage:** Five local files preserve records.

7. Design Diagrams

7.1 Use Case Diagram

The use case diagram represents a generic hostel management user interacting with the six functional areas.

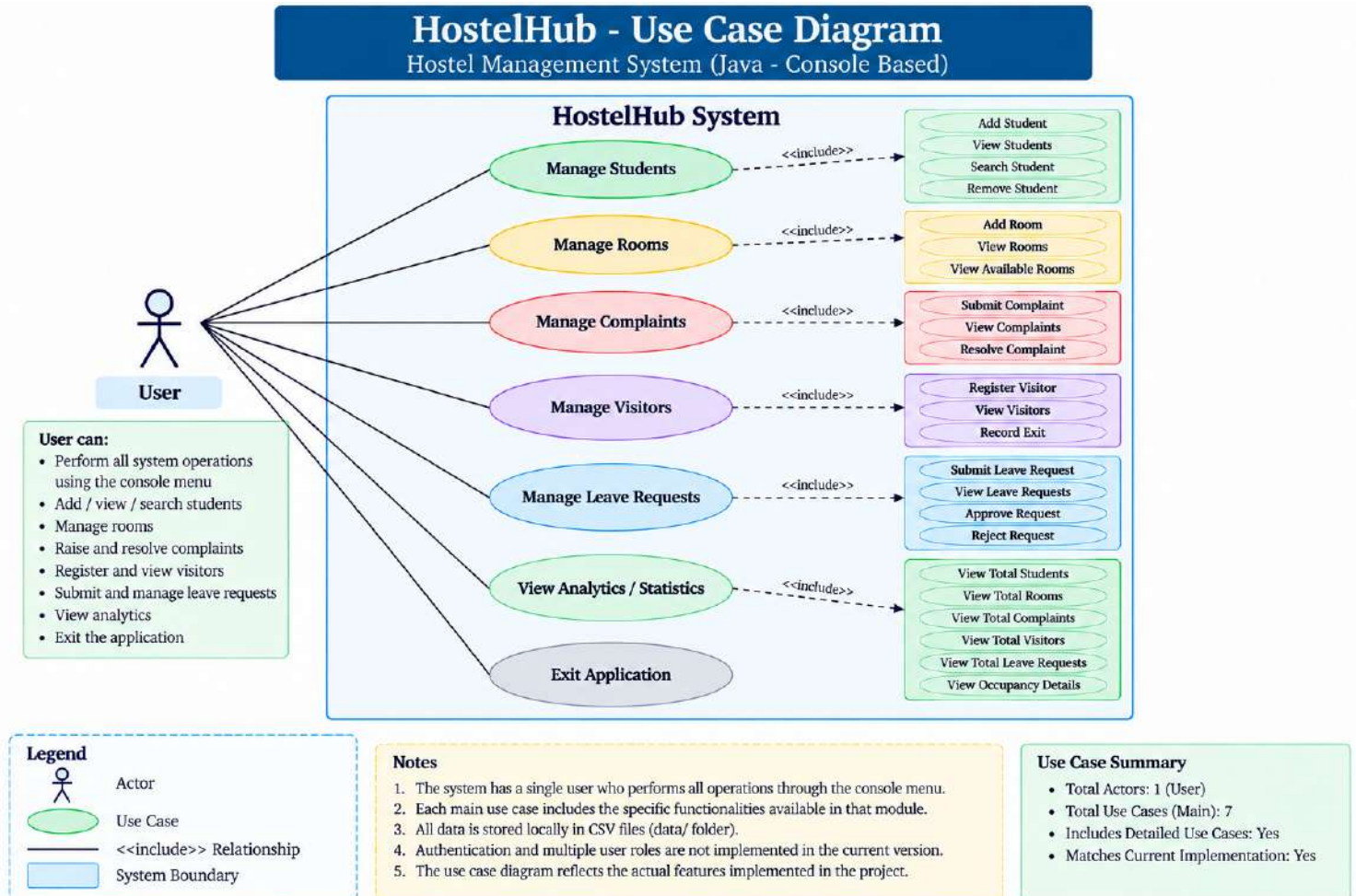


Figure 2. Use case diagram for HostelHub.

7.2 Workflow Diagram

The workflow starts with application launch and menu display. The user selects a module, enters information, the system validates the input, performs the operation, saves updated data and returns to the menu.

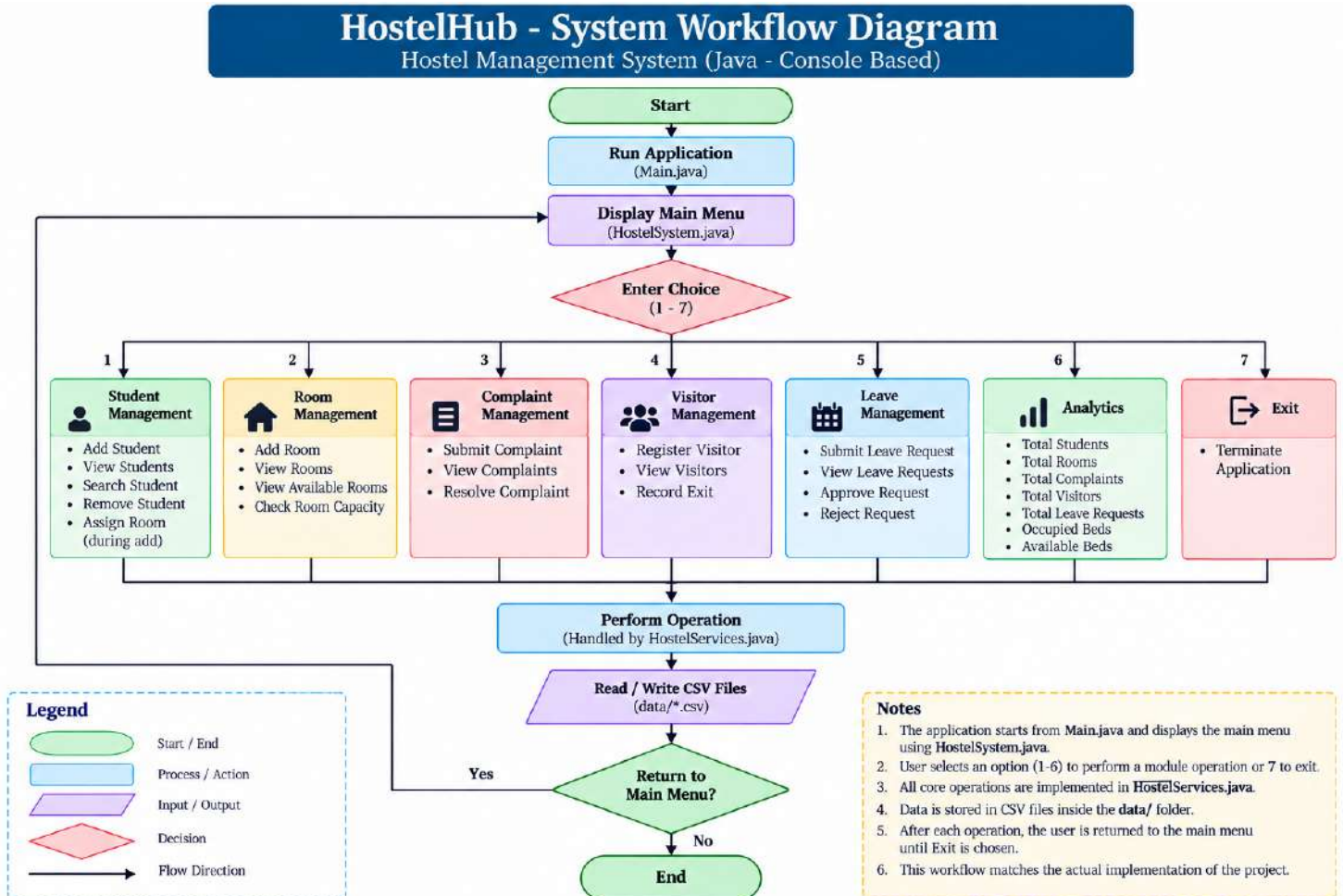


Figure 3. General HostelHub application workflow.

7.3 Sequence Diagram

The sequence below represents a typical record operation involving the user interface, service layer and CSV storage.

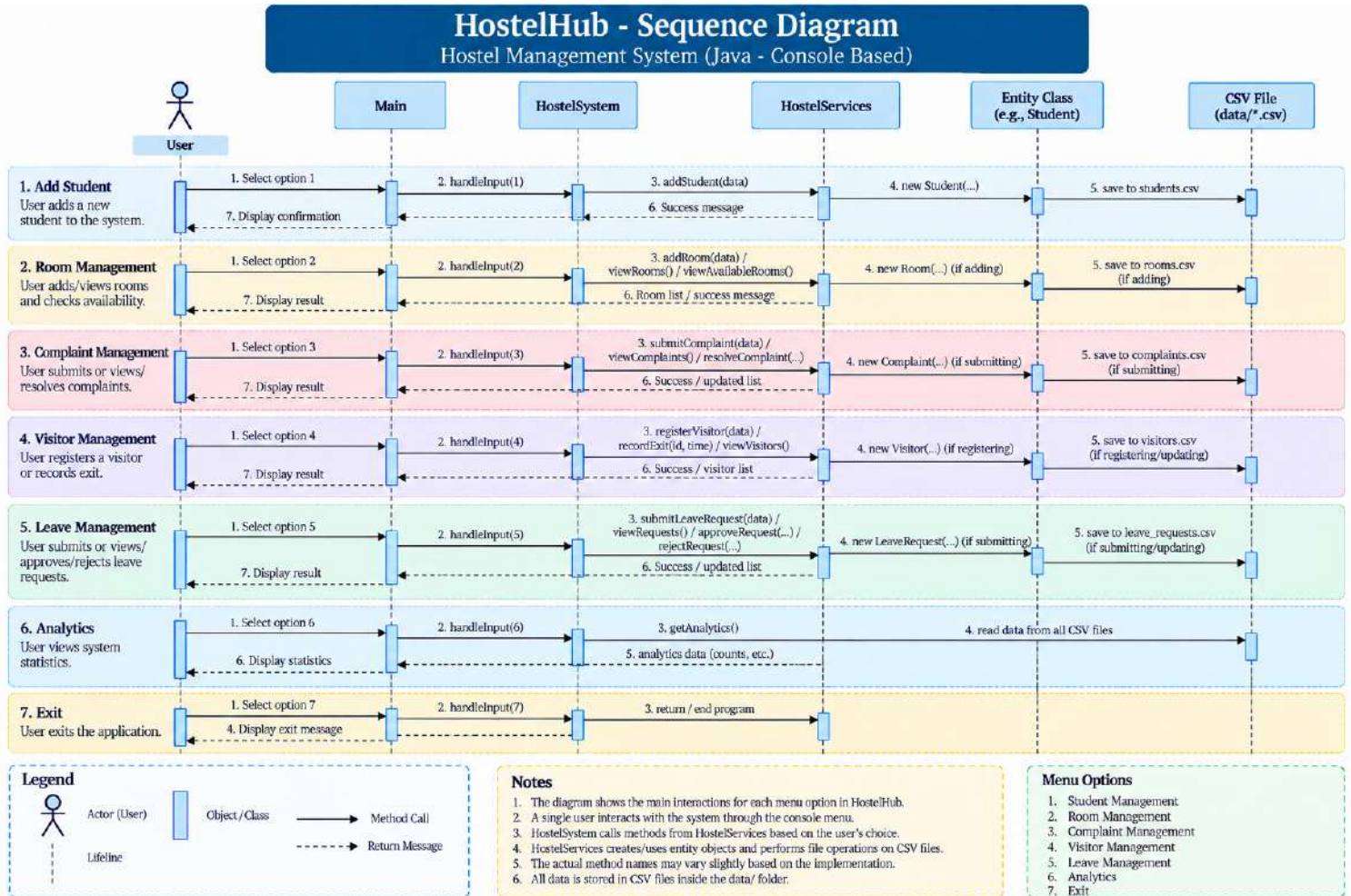


Figure 4. Typical record operation sequence in HostelHub.

7.4 Class Diagram

The class structure reflects the implemented Java source. Main starts the application, HostelSystem manages interaction, HostelServices handles operations, and the five model classes represent stored records.

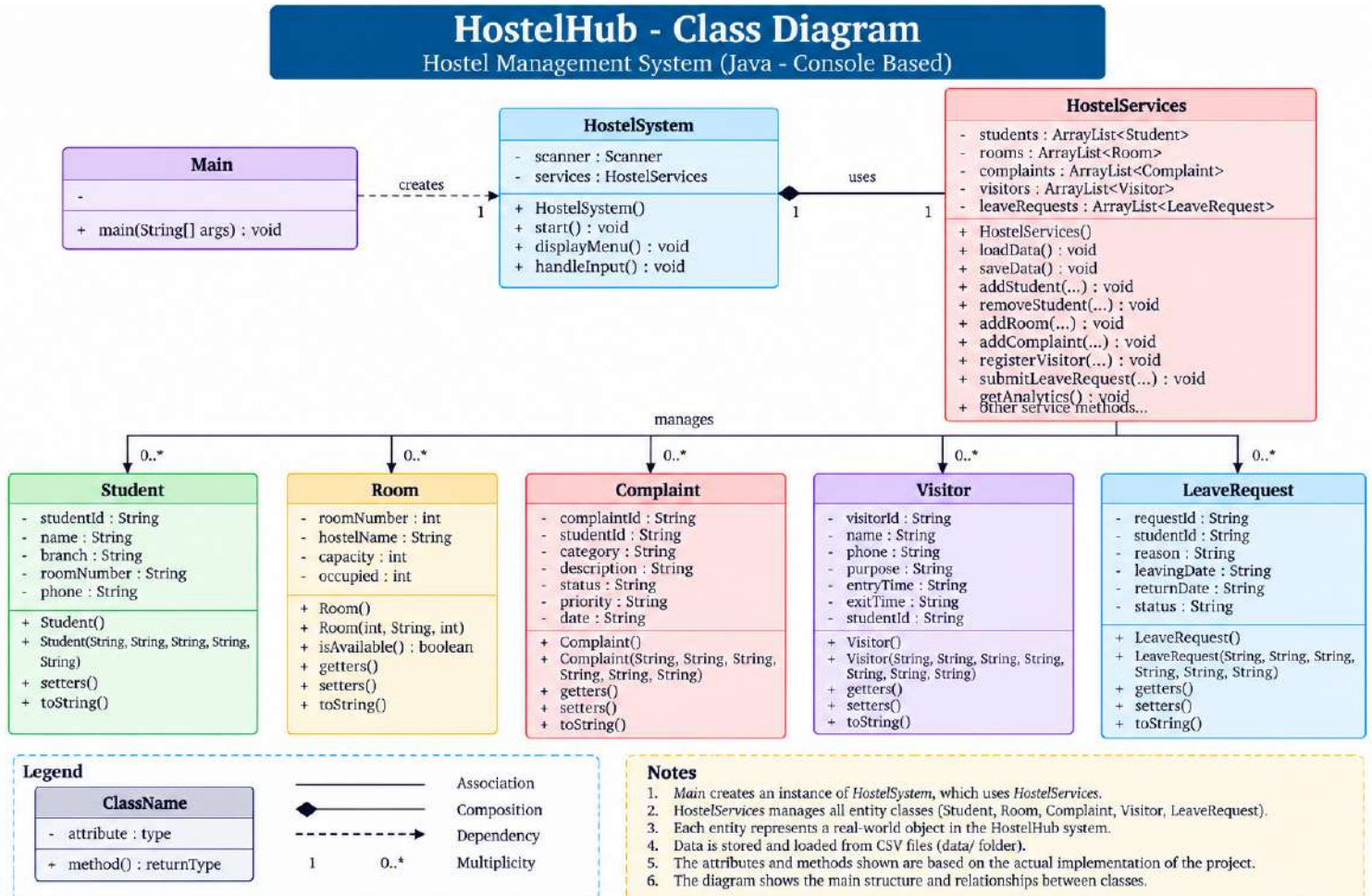


Figure 5. Object-oriented class structure of HostelHub.

7.5 Storage / ER-style Diagram

HostelHub uses CSV files rather than a relational database. This logical storage design shows the records and their references. Student ID is used by complaint, visitor and leave records, while roomNumber represents room assignment.

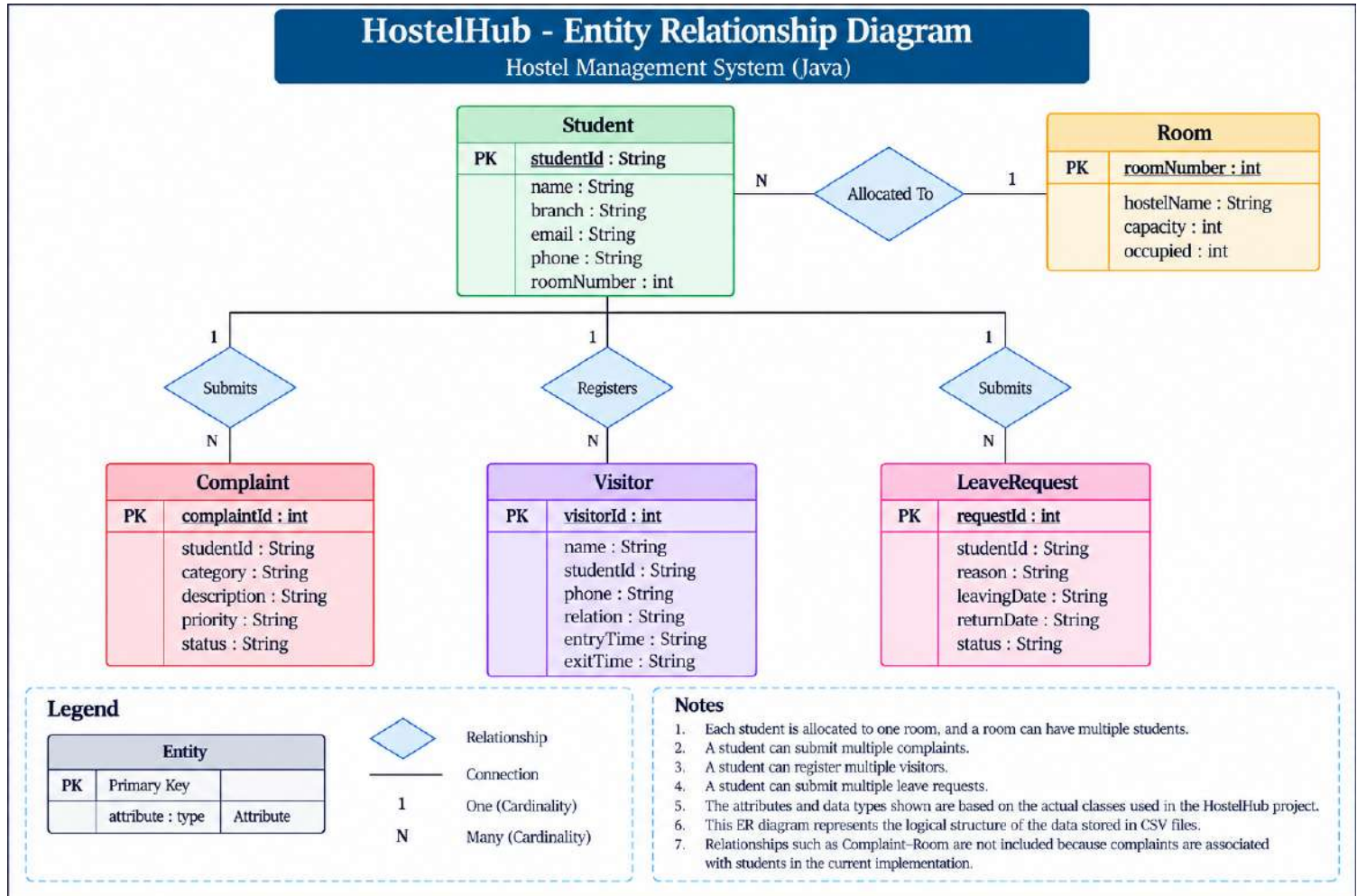


Figure 6. Logical storage design for the CSV-based HostelHub.

8. Design Decisions & Rationale

CSV for Local Persistence

CSV was selected because HostelHub is a lightweight local console application. CSV files are easy to inspect and maintain within the project and do not require an external database server.

Separate Model Classes

Student, Room, Complaint, Visitor and LeaveRequest are separate classes. This keeps related data together and demonstrates encapsulation using private fields and public methods.

HostelServices for Core Operations

HostelServices centralizes searches, record operations, analytics and persistence, while HostelSystem focuses on interaction and navigation.

ArrayList for Record Collections

ArrayList is used for students, rooms, complaints, visitors and leave requests. It provides a straightforward collection structure for the scale of the application.

Validation Before Related Operations

Operations that depend on another record first search for the referenced object. Student allocation requires an existing room with capacity, while complaints, visitors and leave requests require an existing student.

Standalone Testing

The test program uses standard Java code rather than an external testing framework, keeping execution simple and portable.

9. Implementation Details

Main.java	Application entry point.
HostelSystem.java	Menu navigation, input, validation and module interaction.
HostelServices.java	Core operations, searching, analytics and CSV persistence.
Student.java	Student information and CSV conversion.
Room.java	Room capacity, occupancy and student allocation.
Complaint.java	Complaint information and status.
Visitor.java	Visitor information and entry/exit times.
LeaveRequest.java	Leave request information and status.

Data Files

- students.csv — student records and room numbers.
- rooms.csv — room number, hostel name, capacity and occupied beds.
- complaints.csv — complaint ID, student ID, category, description, priority and status.
- visitors.csv — visitor details, student ID and entry/exit times.
- leave_requests.csv — request details, dates and status.

Course Concepts Applied

- Classes, objects and constructors.
- Encapsulation using private fields and public methods.
- Conditional statements, loops and switch-based menu navigation.
- ArrayList and Java Collections.
- File input/output using Java I/O classes.
- Exception handling and input validation.

10. Screenshots / Results

10.1 Main Menu

```
===== HOSTELHUB =====
1. Student Management
2. Room Management
3. Complaint Management
4. Visitor Management
5. Leave Management
6. Analytics
7. Exit
Enter choice: █
```

Figure 7. HostelHub main menu showing the six management modules and analytics option.

The screenshot demonstrates the corresponding HostelHub menu and its available operations.

10.2 Student Management

```
Enter choice: 1

--- Student Management ---
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Back
Enter choice: █
```

Figure 8. Student Management module menu.

The Student Management module provides options to add, view, search and remove student records.

10.3 Room Management

```
Enter choice: 2

--- Room Management ---
1. Add Room
2. View Rooms
3. View Available Rooms
4. Back
Enter choice: █
```

Figure 9. Room Management module menu.

The screenshot demonstrates the corresponding management module and its available operations.

10.4 Complaint Management

```
Enter choice: 3

--- Complaint Management ---
1. Submit Complaint
2. View Complaints
3. Resolve Complaint
4. Back
Enter choice: █
```

Figure 10. Complaint Management module menu.

The screenshot demonstrates the corresponding management module and its available operations.

10.5 Visitor Management

```
Enter choice: 4

--- Visitor Management ---
1. Register Visitor
2. View Visitors
3. Record Exit
4. Back
Enter choice: █
```

Figure 11. Visitor Management module menu.

The screenshot demonstrates the corresponding management module and its available operations.

10.6 Leave Management

```
Enter choice: 5

--- Leave Management ---
1. Submit Leave Request
2. View Leave Requests
3. Approve Request
4. Reject Request
5. Back
Enter choice: █
```

Figure 12. Leave Management module menu.

The Leave Management module supports leave submission, viewing and approval or rejection.

10.7 Analytics

```
Enter choice: 6

--- Hostel Analytics ---
Total Students: 3
Total Rooms: 1
Total Complaints: 1
Total Visitors: 1
Total Leave Requests: 2
Occupied Beds: 3
Available Beds: 0
```

Figure 13. Hostel Analytics output from the captured execution.

The captured analytics output reports 3 students, 1 room, 1 complaint, 1 visitor, 2 leave requests, 3 occupied beds and 0 available beds.

10.8 Test Execution

```
PS D:\HostelHub> javac src\*.java
PS D:\HostelHub> javac -cp src tests\HostelHubTest.java
PS D:\HostelHub> java -cp "src;tests" HostelHubTest
All tests passed.
```

Figure 14. Automated HostelHub test execution output.

The standalone test program completed successfully with the output 'All tests passed.'

11. Testing Approach

Testing was performed using HostelHubTest.java, a standalone Java test class. The tests focus on core model behavior and validation logic.

Test Cases

- **Student:** verifies student ID and room number after object creation.
- **Room:** verifies allocation, occupancy, full-room detection and rejection of an additional allocation.
- **Complaint:** verifies PENDING status and update to RESOLVED.
- **Visitor:** verifies recording of an exit time.
- **Leave request:** verifies PENDING status and update to APPROVED.

Test Execution Commands

```
javac src\*.java
javac -cp src tests\HostelHubTest.java
java -cp "src;tests" HostelHubTest
```

The captured execution completed with the output **All tests passed..** This output is included in Figure 14.

Testing Result

Area	Result
Student model	PASS
Room capacity and occupancy	PASS
Complaint status	PASS
Visitor exit	PASS
Leave request status	PASS

12. Challenges Faced

- Maintaining consistency between student room allocation and room occupancy.
- Handling invalid menu and numeric input without terminating the application.
- Keeping CSV records synchronized with in-memory ArrayList data.
- Converting CSV text into the correct Java object types during loading.
- Organizing multiple hostel operations into separate classes while keeping the console interface understandable.

13. Learnings & Key Takeaways

- Practical use of object-oriented class design and encapsulation.
- Use of Java Collections for maintaining groups of records.
- Use of Java file I/O for persistent local storage.
- Importance of validation when operations depend on existing records.
- Separating user interaction from service operations improves maintainability.
- Testing individual model behaviors helps identify errors before final execution.

14. Future Enhancements

- Add a graphical interface using JavaFX.
- Replace CSV storage with a relational database using JDBC.
- Add user authentication and role-based access control.
- Add search filters and more detailed hostel reports.
- Provide exportable reports in PDF or spreadsheet formats.
- Add notification support for important complaints and leave requests.
- Extend analytics with occupancy trends and historical summaries.

15. References

- Oracle. Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/>
- Herbert Schildt. Java: The Complete Reference, Oracle Press.
- VITyarthi. Build Your Own Project — General Project Instructions & Submission Guidelines, 2026.
- VIT Bhopal University. CSE2006 Programming in Java course syllabus.
- HostelHub project source code, README.md, statement.md and design documentation maintained in the project repository.

Implementation Scope

This report describes the implemented HostelHub version documented in the accompanying source code. The current application is a local console-based Java system using CSV persistence. External databases, graphical interfaces, authentication and notification services are future enhancements rather than current features.